# PARTE 1

## Questão 3

***1. Precisão da tradução:***

o modelo faz a tradução do inglês para o francês conforme ele foi criado, mas pode errar palavras de tuplo significado, termos técnicos e frases mais complexas. A tradução até pode ser fluída e pode atender tarefas simples, porém pode não trazer o significado real da frase ou texto.

***2. Tempo de resposta e desempenho:***

Textos longos e muitos acessos simultâneos aumentam consequentemente o tempo de processamento. Já a velocidade, isso depende do equipamento que está sendo utilizado. Se for um equipamento rubusto, logo a resposta será rápida. Caso seja utilizado um equipamento mais simples, consequentemente a resposta levará mais tempo.

**3. Custo e escalabilidade:**

Execução local não exige pagamento por tradução, mas consome memória e processamento do equipamento utilizado. Dependendo das configurações aplicadas no modelo, pode aumenta a necessidade de memória, e atender mais requisições simultânias pode exigir infraestrutura adicional.

**4. Gírias e contexto:**

Como é um modelo simples e como não há novos treinamentos, gírias, ironias e expressões idiomáticas, por exemplo, podem ser traduzidas de forma literal, não interpretando de fato o que o texto original quer dizer.

## Questão 4

**1. Coerência do texto:**

O modelo GPT-2 utilizado na questão 1 pode até gerar frases bem estruturadas, mas tende a perder o assunto em textos longos, repetir ideias e até mesmo apresentar contradições.

**2. Falhas e incoerências:**

Devido sua obsolecência, o modelo pode inventar informações, confundir fatos e produzir respostas que até parecem corretas, mas não são. Seu funcionamento não garante que o conteúdo seja correto.

**3. Desempenho e latência:**

Textos muito grandes e dependendo das configurações do modelo mais equipamentos com menos recursos tendem a aumentar o tempo de resposta. Então, é necessário avaliar como seu uso será aplicado.

**4. Conteúdo apropriado:**

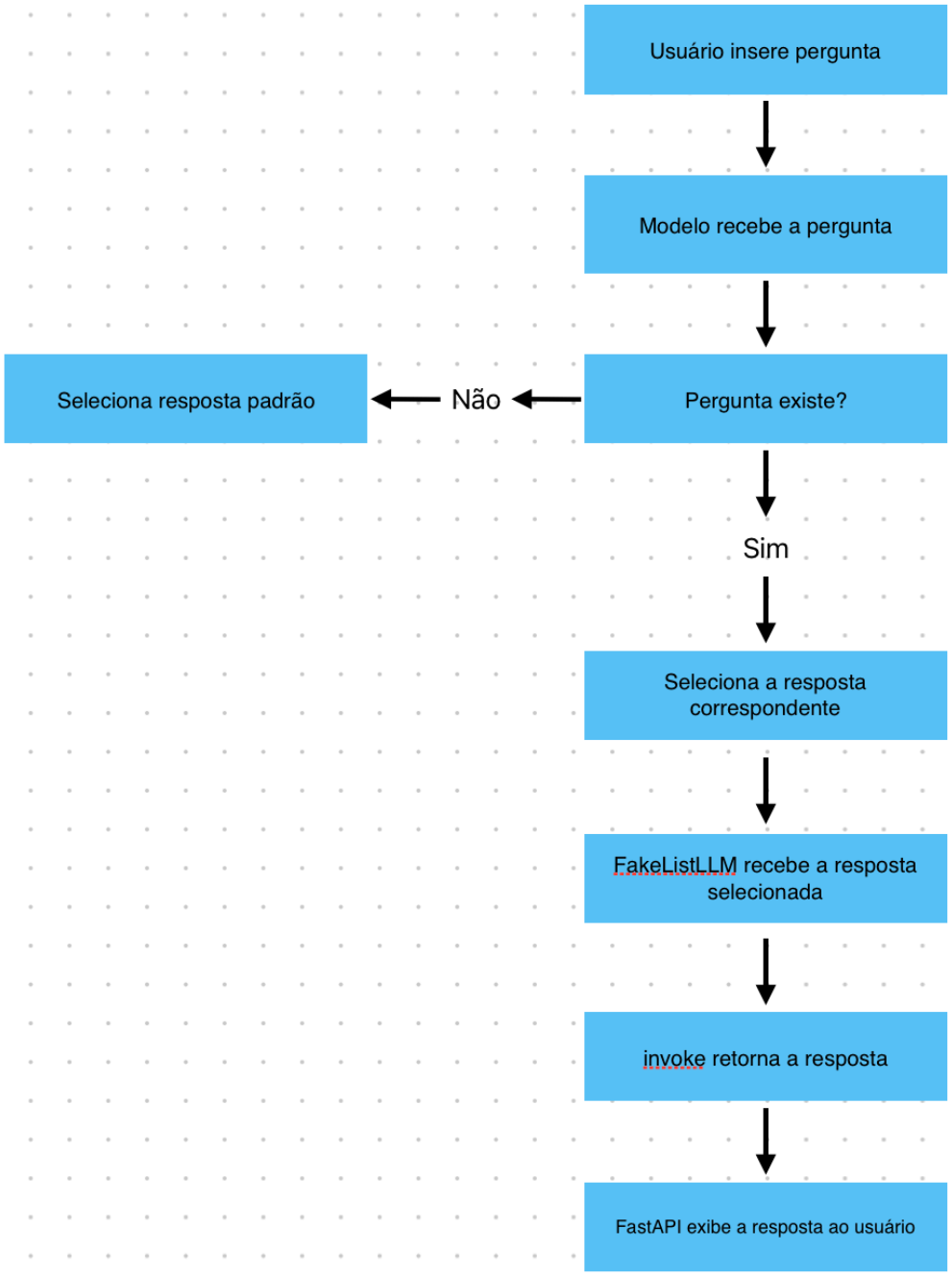
O modelo pode reproduzir preconceitos, linguagem ofensiva e informações inadequadas presentes nos dados de treinamento. O GPT-2 original não possui o mesmo treinamento para seguir instruções e evitar conteúdo prejudicial encontrado em modelos modernos.

---

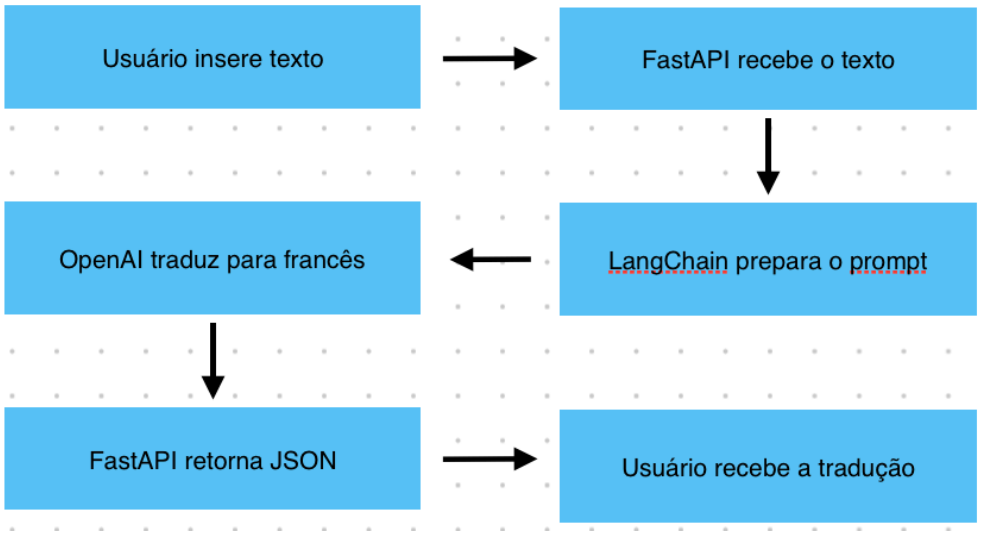
# PARTE 2

**Questão 1**

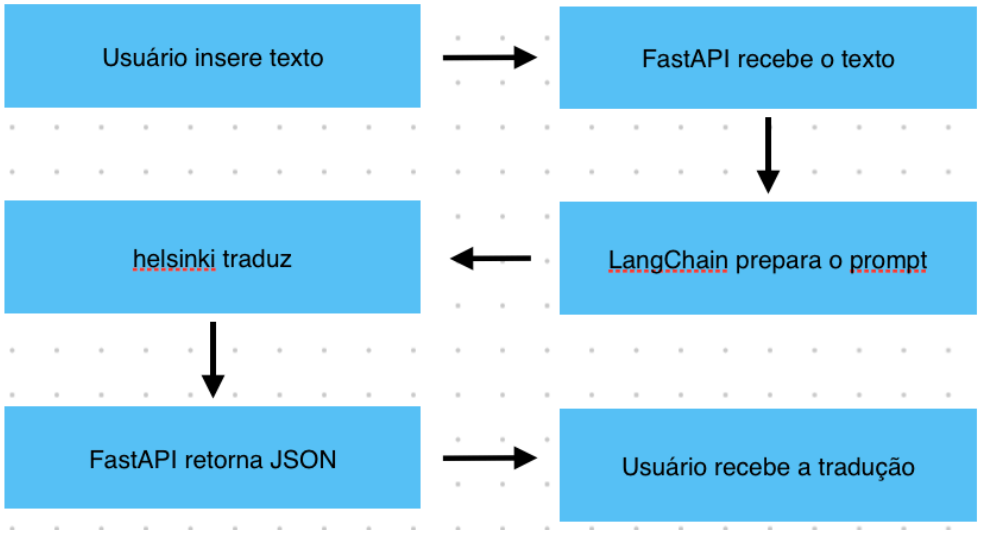
Utilizar o fakeLLM permite testar o funcionamento da aplicação sem precisar utilizar um modelo real. Isso ajudar a ter respostas rápidas sem aguardar que um modelo real gere respostas e sem a necessidade de consumir tokens que são pagos.




**Questão 2**




## Questão 3




## Questão 4

**1. Latência de resposta.**

Cada tradução solicitada é feita uma chamada pela internet e o tempo de geração do modelo. Se forem feitas duas chamadas em sequência, uma para traduzir e outra para revisar, o usuário tem que esperar pelas duas.

**2. Limites de uso.**

A API possui um limite de número de chamadas e de tokens por minuto. Os limites variam conforme o modelo e o nível de uso. Se for utilizado muitas vezes, o número de token se encerra, não permitindo mais seu uso até sua renovação.

**3. Escalabilidade e custo**

Mais traduções e textos mais longos aumentam o consumo de tokens e o custo. Mesmo que o FastAPI consiga receber mais solicitações simultâneas, a capacidade de traduzir continua sujeita aos limites da API.

#### 4. Qualidade da tradução

Este modelo GPT não foi projetado para executar a tarefa de tradução, mas permite ajustes para trazer executar melhor esta tarefa, todavia a tradução pode perder sentido. Quando um modelo é feito para este tipo de tradução, ele tende a executar melhor esta tarefa.

### Questão 5

#### 1. Desempenho e tempo de resposta

O LangChain adiciona uma camada entre a aplicação e o modelo. Em uma chamada simples, o tempo tende a depender mais da execução do modelo do que dessa camada. Se o fluxo fizer várias chamadas em sequência, os tempos se acumulam, demorando para trazer a resposta.

#### 2. Consumo de recursos

Na execução local, é exigido desempenho do equipamento que está sendo utilizado. Caso este equipamento tenha GPU, sua execução se torna meons morosa, desde que não seja feita muitas requisições ou requisições muito grande.

#### 3. Ajuste fino

Alterar max_new_tokens muda o limite de tokens gerados na resposta. Já o fine-tuning modifica os pesos do modelo, por exemplo, treinando-o com pares de frases de um domínio específico. Essa etapa é feita com ferramentas como o Transformers. Depois, o modelo ajustado pode ser integrado ao LangChain.

#### 4. Uso direto da API Hugging Face

Quando a aplicação recebe um texto e devolve a tradução, InferenceClient.translation() permite chamar diretamente um modelo disponível no serviço hospedado. Para manter a execução local, seu pipeline("translation", ...) já resolve. O LangChain faz mais sentido quando a tradução é uma etapa de um fluxo que também resume, classifica ou processa o texto.

### Questão 6

| Abordagem | FastAPI puro, com chamadas diretas | FastAPI com LangChain |
|---|---|---|
| Facilidade de uso e configuração | O pipeline() simplifica o uso do GPT-2 e do Helsinki inglês–alemão. A tradução inglês–francês da Parte 1 exige configurar o tokenizador, o modelo e o dispositivo separadamente. | ChatPromptTemplate e ChatOpenAI organizam a instrução e a chamada ao modelo. A configuração inclui a dependência do LangChain e a chave da API da OpenAI. |
| Latência e desempenho | Nas execuções locais, o tempo de resposta depende muito | Na chamada ao gpt-4.1-mini, o tempo depende principalmente |

| Abordagem | FastAPI puro, com chamadas diretas | FastAPI com LangChain |
|---|---|---|
| | do hardware do equipamento utilizado e tamém do tamanho do texto. O carregamento dos modelos afeta a inicialização da aplicação. | da internet utilizada e do processamento no provedor. O LangChain acrescenta uma etapa de processamento na aplicação. |
| Flexibilidade para diferentes modelos | Dá acesso direto aos parâmetros de geração e ao dispositivo de execução. Ao trocar de modelo, pode ser que seja necessário ser feito mudanças na forma de preparar a entrada e ler a saída. | Facilita combinar prompts, modelos e outras etapas em um fluxo. A troca de modelo ainda pode exigir ajustes nos parâmetros e no formato da resposta, porém é mais simples. |
| Custo e escalabilidade | A execução local consome memória e capacidade de processamento do equipamento. Dependendo do trabalho executado e do equipamento utilizado, o objetivo pode não ser alcançado | No exemplo, a cobrança pelo uso e os limites de requisição vêm da API da OpenAI. A execução remota reduz a carga de processamento no servidor FastAPI, não exigindo processamento local. |
| Adequação para protótipos e produção | É adequado para protótipos e para rotas de produção com uma tarefa bem definida, como receber um texto e retornar a tradução. | É útil em protótipos que testam diferentes prompts ou modelos e em produção quando o fluxo tem várias etapas, como traduzir e depois resumir. |